

Architecture des ordinateurs y Systèmes d'exploitation I



Manuel du 80386

Département d'Architecture des Ordinateurs
M. Jiménez, G. Utrera
Cours 2008-2009 (T1)

Indice

1 Le microprocesseur Intel 80386

1.1 Organisation de la mémoire.....	3
1.2 Registres	
1.2.1 Registres généraux	4
1.2.2 Enregistrements de Segment	5
1.2.3 Registres spéciaux.....	5
1.3 Types de données et stockage.....	6
1.4 Modes d'adressage.....	8
1.4.1 Opérandes immédiates	9
1.4.2 Opérandes dans les registres.....	9
1.4.3 Operands en mémoire	9
1.5 Répertoire des instructions	
1.5.1 Instructions de transfert de données.....	12
1.5.2 Instructions arithmétiques	15
1.5.3 Instructions logiques.....	18
1.5.4 Instructions de contrôle de flux.....	22
1.5.5 Instructions de contrôle du processeur	24
1.5.6 Instructions d'accès aux périphériques d'entrée/sortie.....	25

Le microprocesseur Intel 80386

L'i80386, également connu sous le nom d'i386, est un microprocesseur 32 bits à usage général. Pour réaliser un programme en langage assembleur du i80386, il est nécessaire connaître au préalable les aspects suivants de l'architecture :

- Organisation de la mémoire
- Registres
- Types de données et stockage
- Modes d'adressage
- Répertoire d'instructions

Dans ce chapitre, nous détaillons chacun de ces aspects.

1.1 Organisation de la mémoire

Dans un espace linéaire d'adresses, la mémoire est vue comme une succession de mots numérotés de 0 à une valeur maximale déterminée par la taille du bus de directions. Pour accéder à l'un de ces mots, il faut générer l'adresse associée à ce mot. Le i80386 a un bus d'adresses de 32 bits et, par conséquent, vous pouvez accéder jusqu'à 4 Gigaoctets (2^{32} octets) de mémoire.

Le i80386 propose plusieurs modèles d'organisation de la mémoire qui varient entre les sujets extrêmes :

Le modèle plateau l'espace d'adresses consiste en un seul espace linéaire jusqu'à 4 gigaoctets. Dans ce modèle, une adresse mémoire à niveau assembleur s'identifie par une simple bande de 32 bits.

Le modèle segmenté où l'espace d'adresses consiste en une collection de jusqu'à 16383 espaces linéaires d'adresses de jusqu'à 4 gigaoctets chacun. Dans ce modèle, une adresse mémoire au niveau assembleur consiste en deux composants : un segment (une bande de 16 bits qui identifient un segment) et un déplacement (tire 32 bits qui identifient un octet à l'intérieur d'un segment).

Dans cette matière, nous utiliserons toujours le modèle à plat de mémoire.

1.2 Enregistrements

L'i80386 dispose au total de 16 registres, divisés en trois catégories de base. (voir Figure 1) :

Registres généraux. Il dispose de 8 registres généraux de 32 bits chacun. Ces registres sont principalement utilisés pour stocker des opérandes d'opérations arithmétiques et logiques.

Registres de segment. Dispose de 6 enregistrements de segment de 16 bits chacun. Ces enregistrements ne sont utilisés que dans des modèles de mémoire segmentée. les registres permettent de déterminer, à un instant donné, quel segment de mémoire se est en train de diriger.

Registres spéciaux. Dispose de 2 registres spéciaux de 32 bits chacun. Ces registres sont utilisés pour stocker et modifier certains aspects de l'état de processeur.

%EAX _{0..31}		%AH _{8..15}	%AL _{0..7}	%AX _{0..15}
%EBX _{0..31}		%BH _{8..15}	%BL _{0..7}	%BX _{0..15}
%ECX _{0..31}		%CH _{8..15}	%CL _{0..7}	%CX _{0..15}
%EDX _{0..31}		%DH _{8..15}	%DL _{0..7}	%DX _{0..15}
%EBP _{0..31}				%BP _{0..15}
%ESP _{0..31}				%SP _{0..15}
%ESI _{0..31}				%SI _{0..15}
%EDI _{0..31}				%DI _{0..15}
		%CS _{0..15}		
		%SS _{0..15}		
		%DS _{0..15}		
		%ES _{0..15}		
		%FS _{0..15}		
		%GS _{0..15}		
%EFLAGS				%Drapeaux _{0..15}
0..31				%IP _{0..15}
%EIP _{0..31}				

Figure 1 : Registres du i386

Registres Généraux

Les registres généraux de l'i80386 sont de 32 bits et s'appellent %EAX, %EBX, %ECX, %EDX, %ESP, %EBP, %ESI et %EDI. Chacun de ces registres peut être utilisé pour stocker des opérandes de instructions logiques et arithmétiques et pour stocker des adresses mémoire où se trouvent les opérandes (sauf le %ESP qui ne peut pas être utilisé avec ce second finalité).

La partie basse (les 16 bits faibles) de chacun de ces registres a un nom différent (en concret, %AX, %BX, %CX, %DX, %SP, %BP, %SI et %DI) et peut être utilisée comme un registre de 16 bits. Cette caractéristique est utile pour gérer des données de 16 bits et pour la compatibilité avec les processeurs 8086 et 80286.

De plus, chaque octet des registres %AX, %BX, %CX et %DX a également un nom différent et peut être utilisé comme un registre de 8 bits. Cette caractéristique est utile pour manipuler des caractères et des données de 8 bits. Les noms de ces registres sont %AH, %BH, %CH et %DH pour le byte haut et %AL, %BL, %CL et %DL pour le byte bas.

Tous les registres généraux peuvent être utilisés comme pointeurs en mémoire, comme opérandes de opérations arithmétiques et logiques et pour stocker des résultats. De plus, certains de ceux-ci Les registres sont utilisés de manière implicite dans certaines instructions (%AX et %DX dans la multiplication) et division, %SI et %DI dans les instructions de manipulation de chaîne, etc.).

1.2.2 Registres de segment

Les enregistrements de segment de l'i80386 sont de 16 bits et s'appellent %CS, %SS, %DS, %ES, %FS et %GS. Ces registres ne sont utilisés que dans des modèles de mémoire segmentée et permettent d'identifier chacun des six segments accessibles à un instant donné de l'exécution d'un programme.

Dans un modèle de mémoire plate, qui est celui que nous utiliserons dans notre matière, ces enregistrements de segment ne sont pas utilisés.

1.2.3 Registres spéciaux

Les deux registres spéciaux dont dispose l'i80386 sont de 32 bits chacun et s'appellent %EFLAGS et %EIP. Nous allons ensuite commenter plus en détail chacun de ces registres.

%EFLAGS

Le registre %EFLAGS contrôle certaines opérations et indique l'état du processeur. La partie basse (les 16 bits de poids faible) du %EFLAGS s'appelle %FLAGS et peut être utilisée par le processeur comme une unité. Cette caractéristique est utile lorsque nous exécutons des codes qui ont été conçus pour les processeurs 8086 ou 80286.

Les 16 bits de poids faible du %EFLAGS peuvent être accédés comme une unité et s'appellent %DRAPEAUX.

Chaque bit (également appelé drapeau) du %EFLAGS nous fournit une sorte d'information. Ces drapeaux peuvent être classés en trois groupes : drapeaux d'état, drapeaux de contrôle et drapeaux de système. Les drapeaux du système ne seront pas discutés dans cette publication.

Les indicateurs d'état stockent des informations sur l'état du processeur après exécuter la dernière instruction. De cette façon, le résultat d'une instruction peut influencer l'exécution d'une instruction ultérieure. Les indicateurs d'état sont les suivants :

- **Acarreo. CF (Drapeau de report, bit 0) :** Ce drapeau s'active lorsqu'un acarreo se produit. Dans le bit de poids le plus élevé du résultat de la dernière instruction. Dans le cas contraire, CF se met à zéro.
- **Parité. PF (Drapeau de parité, bit 2) :** Il s'active si le nombre de bits à 1 du byte bas de la dernière instruction est pair. Dans le cas contraire, PF est remis à zéro.
- **Ajuste. AF (Drapeau d'ajustement, bit 4) :** S'active si un report se produit dans le bit 3 (quatrième bit de moindre poids) du résultat de la dernière instruction. Ce drapeau est utilisé dans opérations arithmétiques décimales (chiffres BCD). Dans le cas contraire, AF est mis à zéro.
- **Zéro. ZF (Drapeau Zéro, bit 6) :** S'active si le résultat de la dernière instruction est zéro. Dans un autre cas, ZF passe à zéro.
- **Signo. SF (Drapeau de signe, bit 7) :** Il s'active si le bit de poids le plus élevé du résultat de la dernière instruction est 1 (résultat négatif). Dans le cas contraire, SF est mis à zéro (résultat positif).
- **Dépassement. OF (Drapeau de Dépassement, bit 11) :** Le dépassement dans une opération arithmétique se produit lorsque le résultat de l'opération dépasse la capacité de représentation de l'opérande de destination. Le drapeau OF s'active lorsque cela se produit débordement dans la dernière instruction. Dans le cas contraire, OF est mis à zéro.

Les drapeaux de contrôle stockent des informations de contrôle du processeur. Ces drapeaux sont les suivants :

- Trap. TF (Drapeau de piégeage, bit 8) : Si ce bit est actif, après l'exécution de chaque l'instruction passe à exécuter une routine spéciale. Cette routine pourrait être le noyau d'un depurateur de programmes ou debugger.
- Autorisation d'interruption. SI (Drapeau d'interruption, bit 9) : Si ce bit est actif, le processeur peut être interrompu après l'exécution de l'instruction dans cours.
- Direction. DF (Drapeau de Direction, bit 10) : Les instructions de traitement des chaînes les pointeurs des opérandes s'incrémentent et se décrémentent automatiquement (registres ESI et/ou EDI) après chaque opération élémentaire. Si ce bit est actif, les enregistrements ESI et/ou EDI s'auto-incrémentent. Dans le cas contraire, se autodécrémentent.

Dans le langage machine du i80386, il existe des instructions qui permettent de modifier certains de les indicateurs, les lire, poser des questions à leur sujet, etc. La position relative de chaque indicateur dans le EFLAGS il est montré dans la Figure 1.

%EIP

Le registre %EIP (compteur de programme) stocke toujours l'adresse mémoire où se trouve la следующая инструкция à exécuter. Cet enregistrement ne peut pas être manipulé de forme explicite; se modifie implicitement avec les instructions de contrôle de flux (APPEL, JMP, etc.).

La partie basse (les 16 bits de poids faible) du %EIP s'appelle %IP et peut être utilisée par le processeur comme une unité. Cette caractéristique est utile lorsque nous exécutons des instructions qui ont été conçues pour les processeurs 8086 ou 80286.

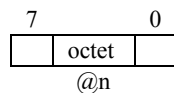
1.3 Types de données et stockage

L'i80386 a trois types de données fondamentales : octets, mots et double mots. Un octet est composé de 8 bits consécutifs stockés à une certaine adresse de mémoire. Les bits sont numérotés de 0 à 7, le bit zéro étant le bit le moins significatif.

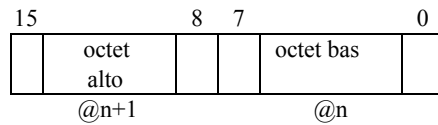
Un mot est composé de deux octets consécutifs, donc il contient 16 bits. Les bits de un mot est numéroté de 0 à 15, le bit zéro étant le moins significatif. Le byte que le bit zéro contient le byte bas et le byte qui contient le bit 15 du mot est le byte haut. Chaque octet d'un mot a sa propre adresse mémoire et est stocké consécutivement en mémoire ; si le byte bas est stocké à l'adresse, le byte alto est à la directionn+1 (voir Figure 2).

Un doubleword est composé de deux mots consécutifs, donc il a 32 bits. Les Les bits d'un double mot sont numérotés de 0 à 31, le bit zéro étant le moins significatif. Le mot qui contient le bit zéro est le mot bas et le mot qui contient le bit 31 est le mot alto. Chacun des quatre octets qui forment un double mot a sa propre adresse de mémoire et sont stockés consécutivement en mémoire ; si l'octet bas du mot bas il est stocké à l'adresse, alors le byte haut du mot bas est à l'adresse n+1, le byte bas du mot haut est en n+2 et le byte haut du mot haut est en n+3.

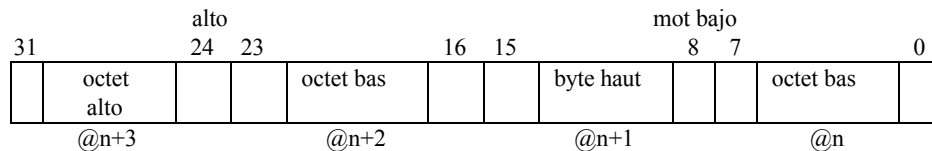
Octet :



Mot



Double mot



Quadmot :

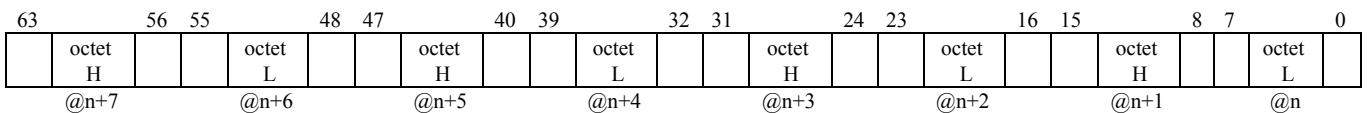


Figure 2 : Types de données et stockage dans l'i80386

Au niveau du langage Assembleur, la taille du résultat d'une instruction est spécifiée au moyen d'un suffixe (b (octet), w (mot) ou l (double mot)) dans le code d'opération de la propre instruction.

Exemples :

ADDB %AH,%AL

ajoute la valeur des enregistrements %AH et %AL. Le résultat est de taille octet.

AJOUTE %AX,%BX

additionne la valeur des registres %AX et %BX. Le résultat est de taille word.

ADDL %EAX,%EBX

additionne la valeur des enregistrements %EAX et %EBX. Le résultat est de taille double mot.

Bien que les octets, mots et double mots soient les types fondamentaux des opérandes, le i80386 prend également en charge des interprétations supplémentaires de ces opérandes. En particulier, l'i80386 offre des instructions au niveau du langage machine pour manipuler et opérer avec les suivants types de données

• Naturels, codés en binaire pur avec :

- 8 bits (octet) : plage de représentation 0 <= x <= 255
- 16 bits (mot) : plage de représentation 0 <= x <= 65535
- 32 bits (double mot) : plage de représentation 0 <= x <= 2³² - 1

- Entiers, codés en complément à 2 avec :
 - 8 bits (octet) : plage de représentation $-128 \leq x \leq 127$
 - 16 bits (mot) : intervalle de représentation $-32768 \leq x \leq 32767$
 - 32 bits (double mot) : plage de représentation $-2^{31} \leq x \leq 2^{31}-1$
- Pointeurs en mémoire
 - 32 bits (double mot) : nous supposons un modèle mémoire plat.
- Bits. Le langage machine offre des instructions pour manipuler des bits à l'intérieur des octets, mots ou doublemots (opérations logiques, décalages et rotations).
- Chaînes. Chaque élément de la chaîne peut être un octet, un mot ou un double mot.
- Chiffres BCD. Un chiffre BCD occupe 4 bits. Deux formats peuvent être utilisés pour les stocker
 - déballés : chaque chiffre BCD est stocké dans la partie basse d'un octet
 - empaquetés : deux chiffres BCD sont stockés dans chaque octet.

Enfin, il convient de mentionner que le transfert de données entre le processeur et la mémoire dans le l'i80386 s'effectue en unités de 32 bits (4 octets) commençant toujours à des adresses de mémoire divisibles par 4. C'est pourquoi il est conseillé (mais pas nécessaire) que les mots et les double mots sont alignés dans des directions paires et divisibles par 4, respectivement. Si les données n'étaient pas alignées, le processeur convertit les requêtes de données désalignées dans la séquence correspondante de requêtes mémoire. Ce désalignement réduit le rendement du processeur car il faut plus de cycles pour les accès à mémoire. Pour obtenir un rendement maximal, les structures de données (y compris la pile) elles doivent être conçues de manière à ce que les mots soient stockés à des adresses paires de mémoire et les double mots dans des adresses divisibles par 4.

1.4 Modes d'adressage

Les modes d'adressage indiquent la localisation des opérandes impliqués dans une instruction. Dans le i386, un opérande peut être dans l'un de ces emplacements :

- Dans l'instruction elle-même (opérande immédiat)
- Dans un registre
- En mémoire

L'accès aux opérandes immédiats ou ceux qui sont dans des registres est plus rapide que l'accès à opérandes en mémoire.

Les instructions du i386 peuvent explicitement avoir un ou deux opérandes. Instructions avec deux opérandes, tels que MOV, ADD, XOR, laissent généralement le résultat sur un des opérandes de la propre instruction, perdant ainsi la valeur originale de l'opérande. Pour ce. Ainsi, dans ce type d'instructions, nous pouvons faire une distinction entre l'opérande source. (qui n'est pas modifié par l'opération) et l'opérande de destination (qui est écrasé avec le au niveau du langage assembleur, la destination est placée à droite et la source à la gauche.

Exemple :

AJOUTE %AX, %BX

ajoute la valeur des registres %AX et %BX, laissant le
résultat en %bx. Le résultat est de taille word.

Dans la plupart des instructions, l'un des deux opérandes (tant la source que le
Le destin peut être en mémoire ou dans un registre. L'autre operande doit être dans un registre ou
être immédiat. Par conséquent, les instructions de deux opérandes de l'i386 seulement
permettent des opérations avec les types d'opérandes suivants :

- enregistrement
- enregistrement
- mémoire
- immédiat
- immédiat

1.4.1 Opérandes immédiats

Certaines instructions utilisent des données de l'instruction elle-même comme opérandes. Ces opérandes
ils s'appellent immédiats et peuvent être de 32, 16 ou 8 bits. Au niveau du langage assembleur, les
valeurs immédiates portent le préfixe \$.

Exemple :

AJOUTE \$4, %AX # ajoute 4 à la valeur du registre %ax.

1.4.2 Opérandes dans les registres

Les opérandes peuvent également être situés dans l'un des registres généraux de 32 bits.
(%EAX, %EBX, %ECX, %EDX, %ESP, %EBP, %ESI et %EDI), dans l'un des registres généraux de 16
bits (%AX, %BX, %CX, %DX, %SP, %BP, %SI et %DI) ou dans l'un des registres généraux de 8 bits
(%AH, %BH, %CH, %DH, %AL, %BL, %CL et %DL).

L'i386 a également des instructions qui manipulent les registres de segment (%CS, %DS, %ES,
%FS, %SS, %GS). Ces instructions ne sont utilisées que dans les modèles de mémoire segmentée.

1.4.3 Opérandes en mémoire

Les opérandes peuvent également être situés en mémoire. Pour le calcul de la
L'adresse effective de mémoire se compose des composants suivants (voir Figure 3) :

- Un déplacement qui se trouve dans l'instruction elle-même. Ce
le déplacement peut être de 8 ou 32 bits.
- Un registre de base
- Un registre d'index qui peut être multiplié par un facteur d'échelle qui
peut être 2, 4 ou 8.

Chacun de ces composants de la direction efficace peut avoir une valeur positive ou négatif. Si la somme des composants dépasse 2^{32} , l'adresse effective est tronquée à 32 bits.

La composante de décalage, qui est codée dans l'instruction elle-même, est utile pour les aspects fixes de la direction de la mémoire, par exemple :

- Localisation des opérandes scalaires.
- Direction de base des vecteurs et des matrices.
- Déplacements de champs à l'intérieur d'une structure.

Les composants base et indice ont des fonctions similaires. Les deux utilisent le même ensemble de enregistrements généraux (sauf que %ESP ne peut pas être utilisé comme registre d'index) et les deux se ils sont utilisés pour des aspects de la gestion de la mémoire qui sont déterminés dynamiquement, par exemple

- Localisation des paramètres et des variables locales dans les sous-routines.
- Le début d'une structure dans un vecteur/matrice de structures.
- Le début d'une dimension dans des vecteurs de plusieurs dimensions.

Le facteur d'échelle permet un accès efficace aux vecteurs/matrices lorsque les éléments del vector/matriz sont de taille 2, 4 ou 8 octets. La multiplication de l'enregistrement d'index par le le facteur d'échelle est effectué par le processeur au même moment où l'adresse est calculée efficace de l'opérateur. De cette façon, il n'est plus nécessaire d'ajouter des instructions qui réalisent cette multiplication.

Au niveau du langage d'assemblage, les opérandes en mémoire sont spécifiés avec le suivant syntaxe :

DÉPLACEMENT (BASE, INDICE, FACTEUR D'ÉCHELLE)

<u>BASE</u>	<u>±</u>	<u>INDICE</u>	<u>*</u>	<u>FACTEUR</u>	<u>D'ÉCHELLE</u>	<u>DÉPLACEMENT</u>
%EAX		%EAX		1		
%ECX		%ECX				
%EDX		%EDX		2		SANS DÉPLACEMENT
%EBX		%EBX				DÉPLACEMENT 8 BITS
%ESP		---		4		DÉPLACEMENT 32 BITS
%EBP		%EBP				
%ESI		%ESI		8		
%EDI		%EDI				

Figure 3 : Calcul de l'adresse effective

Les composants décalage, base et indice peuvent être utilisés dans n'importe quelle combinaison; Tous les composants sont optionnels. Le facteur d'échelle ne peut être utilisé que lorsque on utilise le composant indice. Chaque combinaison possible est utile pour la implémentation de structures de données couramment utilisées dans les langages de haut niveau le langage assembleur. Voici quelques utilisations possibles pour certains des combinaisons des composants d'une adresse :

- **DÉPLACEMENT** : Le déplacement indique uniquement la direction effective du opérant. Cette combinaison est utilisée pour l'accès à des variables scalaires ou à des adresses fixes de mémoire. Exemple :

ADDWvar, %AX

Additionne la variable var avec le contenu du registre %AX. Le
le résultat est de taille mot et est stocké dans %AX.

- **BASE** : l'adresse de l'opérande est stockée dans un registre général. Cette
La combinaison est utile pour l'accès aux vecteurs. Exemple :

ADDW(%EBX), %AX

Ajoute la donnée stockée à l'adresse spécifiée par
%ebx with the register %AX. The result is word-sized and
est stocké dans %AX.

- **BASE + DÉPLACEMENT** : L'opérande se trouve à l'adresse mémoire qui résulte de
somme d'un enregistrement "base" général et d'un déplacement. Cette combinaison est utile pour le
accès aux structures. Exemple :

ADDW-4(%EBX), %AX

Additionne la donnée stockée à l'adresse spécifiée par
%EBX - 4 with the register %AX. The result is of size
le mot y est stocké dans %AX.

- **(indice * FACTOR ÉCHELLE) + DÉPLACEMENT** : Cette combinaison est utile pour accéder à
forma efficace à des vecteurs dont les éléments sont de taille 2, 4 ou 8 octets. La
le composant de décalage stocke l'adresse de base du vecteur et le registre d'index
(multiplié par le facteur d'échelle) indexe le vecteur. Exemple :

ADDWtab(%EBX,4), %AX

Ajoute les données stockées à l'adresse spécifiée par
tab + %EBX * 4 avec le registre %AX. tab est l'adresse de base
d'une variable. Le résultat est de taille mot et reste
stocké dans %AX.

- **BASE + indice + DÉPLACEMENT** : Cette combinaison est utile pour l'accès aux matrices ou à
vecteurs de structures. Exemple :

ADDWtab(%EAX,%EBX), %AX

additionne la donnée stockée à l'adresse spécifiée
par tab + %EAX + %EBX avec le registre %AX. tab est la
adresse de base d'une variable. Le résultat est de taille
Le mot y est stocké dans %AX.

- **BASE + (indice * FACTOR ÉCHELLE) + DÉPLACEMENT** : Permet un accès efficace à
matrices dont les éléments sont de 2, 4 ou 8 octets. Exemple :

ADDWtab(%EAX,%EBX,2), %AX

additionne la donnée stockée à l'adresse spécifiée
par tab + %EAX + %EBX * 2 avec le registre %AX. tab est la
adresse de base d'une variable. Le résultat est de taille
le mot y est stocké dans %AX.

1.5 Répertoire des instructions

Le répertoire d'instructions de l'i386 peut être divisé en les groupes suivants :

- Instructions de transfert de données
- Instructions arithmétiques
- Instructions logiques
- Instructions de contrôle de flux
- Instructions de contrôle du processeur
- Instructions d'accès aux périphériques d'entrée/sortie
- Instructions en virgule flottante
- Instructions multimédias
- Instructions de manipulation de chaînes
- Instructions d'arithmétique BCD

Voici une brève description de certaines des instructions incluses dans le répertoire des instructions de l'i386 (les instructions liées à la manipulation de les chaînes et l'arithmétique BCD ne seront pas détaillées dans ce document).

1.5.1 Instructions de transfert de données

Ces instructions permettent de déplacer des octets, des mots ou des double mots entre la mémoire et les registres. On peut distinguer trois classes différentes dans les instructions de transfert de (1) instructions de transfert de données à usage général, (2) instructions pour la gestion de la pile e (3) instructions pour la conversion de type.

Instructions à usage général

Chaque instruction lorsque cela est pertinent est associée au type de données qu'elle manipule et en ce document sera indiqué de la manière suivante :

nombre_instruction[B|W|L]operands

Cela signifie que selon que l'instruction opère sur des octets, des mots ou des doubles mots, elle devra ajouter à la fin nom_instruction : B, W ou L respectivement.

- MOUV. Copier
MOV[B|W|L]source, destination
source

Copie un octet, un mot ou un double mot de la destination à la source
Drapeaux qui sont modifiés : aucun

Exemple : Copier la valeur 3 (double mot) dans le registre %EAX.

MOVL \$3, %EAX

- XCHG. Échanger

XCHG[B|W|L]opérande1, opérande2

```
tmp:=opérande1
opérande1:=opérande2
tmp
```

```
# Échangez le contenu de opérande1 et opérande2
# Drapeaux qui se modifient : aucun
```

Exemple : Échanger les valeurs des registres %EAX et %EBX.

```
ÉCHANGEZ %EAX,%EBX
```

- LEA. Charger l'adresse effective

LEA[B|W|L]source, destination

```
adresse effective (source)
destination
```

```
# Interprète l'expression source comme une adresse à
# mémoire, chargement du résultat de cette expression à
# destination
# Drapeaux qui se modifient : aucun
```

Exemple : Charger l'adresse %6 + %EAX dans le registre %EDX.

```
LEAL 6(%EAX),%EDX
```

- LAHF. Charger dans AH les indicateurs

LAHF

```
%EFLAGS(SF, ZF, xx, AF, xx, PF, xx, CF)
```

```
Copiez les bits 0-7 du registre des drapeaux dans le registre AH
(cela inclut : AF, CF, PF, SF et ZF)
# Drapeaux modifiés : aucun
```

- SAHF. Charger les drapeaux à partir de AH

SAHF

```
%EFLAGS(SF, ZF, xx, AF, xx, PF, xx, CF):=%AH
```

```
Sauvegarde le contenu du registre AH dans le registre de
# drapeaux.
# Drapeaux qui se modifient : AF CF PF SF ZF
```

Instructions pour la gestion de la pile

- PUSH. Mettre sur la pile

PUSH[B|W|L]source

```
%ESP - taille_opérande_source_en_octets
source
```

Copie dans la pile le contenu de l'opérande source.

```
# Décrémente le registre %ESP selon la taille de l'opérande
# source (2 ou 4 octets).
Flags qui se modifient : Aucune
```

- POP. Prendre de la pile

POP[B|W|L]destination

(%ESP)

%ESP:=%ESP + taille_operande_destination_en_octets

Copiez le contenu de ce qui se trouve en haut de

la pile dans l'opérande de destination. Incrémente le registre %ESP

selon la taille de l'opérande de destination (2 ou 4 octets).

Drapeaux qui ne sont pas modifiés : Aucuns

- PUSHF. Mettre dans la pile le registre des drapeaux

PUSHF[W|L]

%ESP-taille_enregistrement_flags

(%ESP):=enregistrement_des_drapeaux # %EFLAGS ou %FLAGS

Copiez le contenu du registre de drapeaux dans la pile (2 ou

4 octets)

Drapeaux qui sont modifiés : Aucun

- POPF. Prendre de la pile le registre des indicateurs

POPF[W|L]

registro_de_flags :=(%ESP)

%ESP + taille_enregistre_flags

Copiez le contenu de ce qui se trouve en haut de

la pile dans le registre de drapeaux. Incrémente le registre

%ESP selon la taille de l'opérande destination (2 ou 4 octets).

Drapeaux qui se modifient : Tous

Instructions pour la conversion de type

- CBW. Convertir un octet en mot avec extension de signe

CBW

%AX:=%AL (avec extension de signe)

Convertit le byte en %AL en un mot (deux bytes) en %AX

étendant le signe de %AL en %AH

Drapeaux qui se modifient : Aucun

- CWDE. Convertir un mot en double mot avec extension de signe

CWDE

%EAX:=%AX (avec extension de signe)

Convertit un mot (2 octets) en %AX en un double mot (4

octets) en %EAX, étendant le signe de %AX dans %EAX

Drapeaux qui sont modifiés : Aucun

- CWD. Convertir un mot en double mot avec extension de signe

CWD

%DX:%AX:=%AX (avec extension de signe)

Convertit un mot (2 octets) en %AX en un double mot (4 octets)

octets) en %DX:%AX, étendant le signe de %AX en %DX

Drapeaux que l'on modifie : Aucun

- CDQ. Convertir un double mot en quad mot (64 bits) avec extension de signe

CDQ

`%EDX:%EAX:=%EAX` (avec extension de signe)

Convertit un double mot (4 octets) dans %EAX en un quad mot (8 octets) en %EDX, étendant le signe de %EAX en %EDX
Drapeaux qui sont modifiés : Aucun

- MOVZX. Copier et convertir un octet/mot en mot/double mot AVEC extension de signe

MOVZX source, destination

`source (with sign extension)`

Copie le contenu de l'opérande source dans la destination
étendre le signe.
Drapeaux qui se modifient : Aucun

- MOVZX. Copier et convertir un octet/un mot en mot/double mot SANS extension de signe

MOVZX source, destination

`destination:=source (sans extension de signe, ajoute des zéros)`

Copie le contenu de l'opérande source dans la destination
ajout de zéros.
Drapeaux qui se modifient : Aucun

1.5.2 Instructions arithmétiques

Ces instructions manipulent des données numériques codées en binaire. Ces instructions offrent les opérations standards d'addition, de soustraction, de multiplication et de division, ainsi que incrémenter, décrémenter, comparer et changer de signe.

Ces instructions arithmétiques opèrent sur des entiers avec et sans signe. Les instructions modifient les indicateurs ZF, CF, SF et OF de telle sorte que les instructions suivantes puissent interpréter le résultat comme un entier avec ou sans signe. CF contient des informations pertinentes pour les entiers non signés ; SF et OF contiennent des informations pertinentes pour les entiers signés ; et ZF contient des informations pertinentes pour les entiers avec et sans signe.

- AJOUTER. Résumé

AJOUTER[B|W|L]source, destination

`destination := destination + source`

Ajoute le contenu de l'opérande source à celui de destination, remplaçant le contenu de ce dernier par le résultat.
Drapeaux qui sont modifiés : AF CF OF PF SF ZF

- ADC. Somme avec report

ADC[B|W|L]source, destination

destino:=destino + fuente + CF

Ajoute le contenu de l'opérande source à celui de destination,

remplaçant le contenu de ce dernier par le résultat. Si

CF est à 1, on ajoute 1 au résultat. Avec cette instruction, on
Vous pouvez effectuer des sommes de nombres de plus de 32 bits.

Drapeaux qui sont modifiés : AF CF OF PF SF ZF

- INC. Augmenter

INC[B|W|L]destino

destino+=1

Ajoute 1 au contenu de l'opérande de destination, remplaçant le

contenu de ce dernier avec le résultat.

Drapeaux qui se modifient : AF OF PF SF ZF

- SUB. Restar

SUB[B|W|L]source, destination

destino = destino - source

Resta le contenu de l'opérande source à celui de destination,

remplaçant le contenu de ce dernier par le résultat.

Drapeaux modifiés : AF CF OF PF SF ZF

- SBB. Restar avec transport

SBB[B|W|L]source, destination

destino=fuente-CF

Resta le contenu de l'opérande source à celui de destination,

remplaçant le contenu de ce dernier par le résultat. Si

CF est à 1, on soustrait 1 au résultat.

Flags qui se modifient : AF CF OF PF SF ZF

- Décrémenter

DEC[B|W|L]destino

destino:=destino-1

Soustraire 1 au contenu de l'opérande de destination, en remplaçant le

contenu de ce dernier avec le résultat.

Drapeaux qui sont modifiés : AF OF PF SF ZF

- NEG. Changer le signe en complément à 2

NEG[B|W|L]destino

destino = - destino

Resta l'opérande de destination à 0, et lui applique le complément à 2,

remplaçant l'opérande de destination par le résultat.

Drapeaux qui se modifient : AF CF OF PF SF ZF

- **CMP**

CMP[B|W|L]opérande1, opérande2

operando2 - operando1

Soustraire le contenu de l'opérande1 du contenu de l'opérande2
 # sans stocker le résultat.
 # Drapeaux modifiés : AF CF OF PF SF ZF

- **MUL. Multiplier des entiers non signés**

MUL[B|W|L]source

si source est un octet alors

%AX:=%AL * source

fsi

si source est un mot alors

%DX:%AX:= %AX * source

fsi

si source est un double mot alors

%EDX:%EAX:= %EAX * source

fsi

Multiplie le contenu de l'opérande source par le contenu
 # de %AL, %AX ou %EAX en stockant le résultat dans %AX, %DX ou
 # %EDX:%EAX en fonction de si l'opérande source est un octet,
 # mot ou double mot respectivement.
 # Drapeaux à modifier : CF OF (AF, PF, SF, ZF indéfinis)

- **IMUL. Multiplier des entiers signés**

IMUL[B|W|L]source

si source est un octet alors

%AX:=%AL * source

fsi

si la source est un mot alors

%DX:%AX:= %AX * source

fsi

si la source est un double mot alors

%EDX:%EAX:= %EAX * source

fsi

destino *= fuente

Multipliez le contenu de l'opérande source par le contenu
 # de %AL, %AX ou %EAX stockant le résultat dans %AX, %DX ou
 # %EDX:%EAX selon que l'opérande source est un octet,
 # mot ou double mot respectivement.

IMUL[B|W|L]source, destination

destino *= fuente

IMUL[B|W|L]source1, source2, destination

destinatio=source1 * source2

#Drapeaux qui se modifient : CF OF (AF, PF, SF, ZF indéfinis)

- DIV. Diviser des entiers non signés

DIV[B|W|L]source

si source est un octet alors

```
%AL:= quotient (%AX / source)
reste (%AX / source)
```

fsi

si la source est un mot alors

```
%AX:= quotient (%DX:%AX / source)
resto (%DX:%AX / fuente)
```

fsi

si la source est un double mot alors

```
%EAX:= quotient (%EDX:%EAX / source)
%EDX:= resto (%EDX:%EAX / source)
```

fsi

```
# Divisez le contenu %AL, %AX ou %EAX par le contenu de
# opérant la source en stockant le quotient et le reste dans %AL et %AH ou
# en %AX et %DX ou en %EAX et %EDX selon que l'opérande
# la source est un octet, un mot ou un double mot respectivement.
# Drapeaux qui sont modifiés : (AF, CF, OF, PF, SF, ZF indéfinies)
```

- IDIV. Diviser les entiers signés

IDIV[B|W|L]source

si la source est un octet alors

```
%AL:= quotient (%AX / source)
resto (%AX / fuente)
```

fsi

si fuente est un mot alors

```
= quotient (%DX:%AX / source)
resto (%DX:%AX / fuente)
```

fsi

si fuente est un doubleword alors

```
%EAX:= quotient (%EDX:%EAX / source)
%EDX := resto (%EDX:%EAX / source)
```

fsi

```
# Diviser le contenu %AL, %AX ou %EAX par le contenu de
# opérant la source en stockant le quotient et le reste dans %AL et %AH ou
# en %AX et %DX ou en %EAX et %EDX selon que l'opérande
# fuente est un octet, un mot ou un double mot respectivement.
# Drapeaux qui se modifient : (AF, CF, OF, PF, SF, ZF indéfinis)
```

1.5.3 Instructions logiques

Le groupe d'instructions logiques opère au niveau des bits et inclut des opérations de type booléen, déplacements et rotations.

- NON. Négation logique

NOT[B|W|L]destino

NON destino

```
# Réalisez le complément à 1 du contenu de l'opérande destination.
# Drapeaux qui ne sont pas modifiés : Aucun
```

- ET. Y logique

ET[B|W|L]source, destination

destino := destino ET fuente

Effectue l'AND bit à bit entre les opérandes source et destination,
remplaçant le contenu de ce dernier par le résultat.
Drapeaux qui se modifient : CF OF PF SF ZF (AF indéfini)

- OU. O logique

OU[B|N|L]source, destination

destination := destination OU source

Réalise l'OR bit à bit entre les opérandes source et destination,
remplaçant le contenu de ce dernier par le résultat.
Drapeaux qui se modifient : CF OF PF SF ZF (AF indéfini)

- XOR. L'exclusif logique

XOR[B|W|L]source, destination

destin := destin XOR source

Effectue l'opération XOR bit à bit entre les opérandes source et destination,
remplaçant le contenu de ce dernier par le résultat.
Flags qui se modifient : CF OF PF SF ZF (AF indéfinie)

- TEST. Y logique

TEST[B|W|L]source, destination

destination

Réalise l'AND bit à bit entre les opérandes source et destination,
sans stocker le résultat.
Drapeaux modifiés : CF OF PF SF ZF (AF indéfini)

- SAL / SHL. Déplacement (arithmétique / logique) vers la gauche

SAL[B|W|L]comptable, destination

SHL[B|W|L]compteur, destination

```
pour j:=1 à contador faire
    CF:=destination (bit size_destination_in_bits-1)
    pour i:=taille_cible_en_bits-1 jusqu'à 1 faire
        destin (bit i) := destin(bit i-1)
        destino (bit 0):=0
    finpour
finpour
```

Déplace les bits de l'opérande de destination vers la gauche autant de
positions as indicated by the counter operand. From the right
entran ceros. Le drapeau de retenue contient le dernier bit qui
reste hors de l'opérande de destination.
L'opérande compteur doit être : %CL ou un opérande immédiat.
Drapeaux qui se modifient : CF OF PF SF ZF (AF indéfini)

- SHR. Décalage (logique) à droite

SHR[B|W|L]contador, destination

```

pour j:=1 à contador faire
    destino (bit 0)
    pour i:=0 à taille_operande_en_bits-2 faire
        destino (bit i) := destino(bit i+1)
        destin (bit taille_opérande_en_bits-1) := 0
    finpour
finpour

```

Déplacez vers la droite les bits de l'opérande de destination autant de
positions comme indiqué par l'opérande compteur. Par la gauche
entrer des zéros. Le drapeau de report contient le dernier bit qui
reste en dehors de l'opérande de destination.

L'opérande compteur doit être : %CL ou un opérande immédiat.

Drapeaux modifiés : CF OF PF SF ZF (AF indéfinie)

- SAR. Décalage (arithmétique) à droite

SAR[B|W|L]compteur, destination

```

pour j:=1 à compteur faire
    destination (bit 0)
    pour i:=0 à taille_operande_en_bits-2 faire
        destino (bit i):=destino(bit i+1)
        destino(taille_opérando_en_bits-1):=
            destino(bit taille_operande_en_bits-2)
    finpour
fin de boucle

```

Décale à droite les bits de l'opérande de destination autant
positions comme indiqué par l'opérande compteur. Par la gauche
se replica le bit de signe. Le drapeau de transfert contient le
dernier bit qui reste en dehors de l'opérande de destination.

L'opérande compteur doit être : %CL ou un opérande immédiat.

Drapeaux modifiés : CF OF PF SF ZF (AF indéfini)

- ROL. Rotation à gauche

ROL[B|W|L]compteur, destination

```

pour j:=1 à contador faire
    tmp:=destination (bit size_operand_in_bits-1)
    pour i:=taille_opérande_en_bits-1 à 1 faire
        destin (bit i):=destin(bit i-1)
        tmp
    fin
finpour

```

Rota vers la gauche les bits de l'opérande de destination autant
positions as indicated by the counter operand. The bits that
sortent par la gauche et réentrent par la droite. Le drapeau de
Le report contient la valeur du dernier bit tourné.

L'opérande compteur doit être : %CL ou un opérande immédiat.

Flags qui se modifient : CF OF

- ROR. Rotation à droite

ROR[B|W|L]compteur, destination

```

pour j:=1 à contador faire
    tmp:=destination (bit 0)
    pour i:=0 à taille_operande_en_bits-2
        destino (bit i) := destino (bit i + 1)
        destination (bit taille_operande_en_bits-1):=tmp
    finpour
finpour

```

Faites pivoter vers la droite les bits de l'opérande destination autant de fois que

```

# positions comme indiqué par l'opérande compteur. Les bits que
# sortent par la droite, rentrent par la gauche. Le drapeau de
Le report contient la valeur du dernier bit tourné.
# L'opérande compteur doit être : %CL ou un opérande immédiat.
# Drapeaux qui se modifient : CF OF

```

- RCL. Rotation à gauche avec report

RCL[B|W|L]compteur, destination

```

pour j:=1 à contador faire
    tmp:=CF
    CF=destination (bit operand_size_in_bits-1)
    pour i:=taille_operande_en_bits-1 jusqu'à 1 faire
        destino (bit i):=destino(bit i-1)
        tmp
    finpour
finpour

```

Faire pivoter vers la gauche les bits de l'opérande destination autant de fois
positions comme indiqué par l'opérande compteur. Les bits qui
sortent par la gauche, réentrent par la droite, en passant par
le drapeau de Carry.

L'opérande compteur doit être : %CL ou un opérande immédiat.

Drapeaux modifiés : CF OF

- RCR. Rotation à droite avec report

RCR[B|W|L]compteur, destination

```

pour j:=1 à contador faire
    tmp:=CF
    destino (bit 0)
    pour i:=0 à taille_operande_en_bits-2
        destino (bit i) := destino(bit i+1)
        destin (bit taille_opérande_en_bits-1):=tmp
    finpour
finpour

```

Faites pivoter à gauche les bits de l'opérande de destination autant de fois
positions comme indiqué par l'opérande compteur. Les bits qui
sortent par la gauche, réentrent par la droite, passant par
la flag de Carry.

L'opérande compteur doit être : %CL ou un opérande immédiat.

Drapeaux qui se modifient : CF OF

1.5.4 Instructions de contrôle de flux

Le i80386 offre des instructions de contrôle pour diriger le flux d'exécution. Ces instructions permettent de modifier le séquençage implicite des programmes (qui exécutent les instructions dans le même ordre où elles sont stockées en mémoire). Les instructions de contrôle de flux peuvent être conditionnelles ou inconditionnelles. Les conditionnelles consultent les indicateurs du processeur et modifient le séquençage implicite en fonction de ses valeurs. Les instructions inconditionnelles modifient toujours le séquençage implicite.

- APPELER. Appel de sous-routine

APPELER étiquette

```
POUSSEZ %EIP
étiqueta
```

```
# Invoquer la sous-routine en sauvegardant l'adresse sur la pile
retour de la routine actuelle.
# Drapeaux qui modifient : Aucun
```

- RET. Retour de sous-routine

RET

```
POP %EIP
```

```
Retourne à l'instruction pointée par la première valeur qui se
trouve dans la pile au moment de l'exécution de cela
# instruction.
# Drapeaux qui se modifient : Aucun
```

- JMP. Saut inconditionnel

JMP étiquette

```
étiquette
```

```
Saut inconditionnel à la première instruction après
# étiquette.
# Drapeaux qui ne se modifient pas : Aucun
```

- Jxx. Sauts conditionnels

Jxxétiqueta

```
Si la condition spécifiée est remplie alors :
étiquette
```

```
Saut conditionnel à l'étiquette en fonction de si la condition
La condition spécifiée dans l'instruction est vraie.
# dépendra de l'état des drapeaux au moment de la
exécution de l'instruction de saut.
# Drapeaux qui se modifient : Aucun
```

Exemple d'instructions liant des valeurs sans signe (nous supposons que la instruction précédente au saut est CMPb, qui effectue en interne la soustraction a-b):

JA étiquette	salto siasuperior ab	(CF=0 et ZF=0)
Étiquette JAE	salto siasuperior ou égal ab	(CF=0)
JB etiq	saut si inférieur de	(CF=1)
Étiquetage JBE	salto si inférieur ou égal à ab	(CF=1 ou ZF=1)
JNBE étiquette	salto siasuperior ab	(CF=0 et ZF=0)
Étiquette JNB	salto siasuperior ou égal ab	(CF=0)
JNAE étiquette	salto si inférieur ab	(CF=1)
JNA etiquette	salto si inférieur ou égal à ab	(CF=1 ou ZF=1)

Exemple d'instructions liant des valeurs avec un signe (nous supposons que la l'instruction précédente au saut est CMPb, qui effectue en interne la soustraction b):

JG étiquette	salto siamayor queb	(SF=OF et ZF=0)
JGE étiquette	salto siamayor ou égal à b(SF=OF)	
JL etiq	salto siamenor queb(SF!=OF)	
JLE etiq	saut si inférieur ou égal à b(SF != OF ou ZF = 1)	
JNLE etiq	salto siamayor queb	(SF=OF et ZF=0)
Étiquette JNL	salto siamayor o igual que b(SF=OF)	
JNGE etiq	salto siamenor queb	(SF!=OF)
Étiquette JNG	salto si inférieur ou égal à b (SF != OF ou ZF = 1)	

Exemple d'instructions de saut en fonction des indicateurs :

JE ou JZ étiquette	salto si égal	(ZF=1)
JNE ou JNZ étiquette	salto si non égal	(ZF=0)
JC etiq	salto s'il y a un transport	(CF=1)
Étiq JNC	salto s'il n'y a pas de report	(CF=0)
JO etiq	salto s'il y a débordement	(OF=1)
JNO etiq	saut si pas de débordement (OF=0)	
Étiquette JS	saut si négatif	(SF=1)
Étiquette JNS	salto si positif	(SF=0)
JP ou JPE étiquette	salto si paridad	(PF=1)
JNP ou JPO étiquette	saut si pas de parité	0

Exemple d'instructions de saut en fonction d'un registre :

JCXZ etiq salto si registre %CX est zéro
JECXZ étiquette saut si registre %ECX est zéro

• INT. Interruption logiciel

ENT n

```
PUSH %EFLAGS
0
SI:=0
PUSH %EIP
```

```
# Stocke automatiquement dans la pile le registre des indicateurs
# (%EFLAGS) and the return address (%EIP), disables
# interruptions et traps et initie une interruption logiciel.
# Drapeaux qui sont modifiés : TF IF (sont mis à 0)
```

- IRET. Retour d'interruption

IRET

```
POP %EIP
POP %EFLAGS
```

Restaure la valeur de %EIP et le registre des indicateurs (%EFLAGS) depuis la pile et retourne à l'instruction pointée par %EIP.
Drapeaux modifiés : AF CF DF IF PF SF TF ZF

1.5.5 Instructions de contrôle du processeur

Le i80386 offre des instructions de contrôle du processeur. Ces instructions permettent modifier l'état des indicateurs du mot d'état (%EFLAGS).

- CLC. Effacer le drapeau de transport

CLC

```
CF:=0
```

Met à 0 le drapeau de retenue.
Drapeaux qui se modifient : CF

- STC. Activer le drapeau de transport

STC

```
1
```

Met à 1 le drapeau de retenue
Drapeaux qui se modifient : CF

- CMC. Complément de drapeau de transport

CMC

```
CF := non CF
```

Changez le drapeau de retenue par son complément à 1.
Drapeaux qui se modifient : CF

- CLI. Efface le drapeau d'interruption

CLI

```
SI:=0
```

Met à 0 le drapeau d'interruption
Drapeaux qui se modifient : IF

- STI. Activer le drapeau d'interruption

STI

```
SI:=1
```

Met le drapeau d'interruption à 1.
Drapeaux qui se modifient : IF

1.5.6 Instructions d'accès aux périphériques d'entrée/sortie

Le i80386 offre des instructions d'accès aux ports d'entrée/sortie du processeur pour le transfert de données entre les périphériques et le processeur.

- IN. Lire un port E/S

IN[B|W|L]puerto_E/S, accumulateur
(puerto_E/S)

Lire un octet, un mot ou un double mot depuis le port_E/S et le stocker
dans l'accumulateur (%AL, %AX ou %EAX respectivement). Si le nombre
de port est dans la plage 0-255 peut être spécifié comme un
opérande immédiat, sinon cela doit être précisé dans
l'enregistrement %DX.

- SORTIE. Écrire dans un port d'E/S

OUT accumulateur, port_E/S
accumulateur

Le byte, word ou double mot qui est dans l'accumulateur (%AL,
%AX ou %EAX respectivement) est écrit dans le port_E/S
Si le numéro de port est dans la plage 0-255, il peut
se spécifier comme un opérande immédiat, sinon
doit être spécifié dans le registre %DX.